KI-bruk: oppdatering av kategori

Dokumentasjon av hvordan KI ble brukt da adminsiden fikk oppdatering av kategori per spill i tabellen, med PUT /api/games/{id}

Hva KI ble brukt til

Oppdateringen skulle bygges om fra et eget skjema til et felt og en lagreknapp på hver rad i tabellen. Prosjektet ble delt med KI (Claude), med en forespørsel om å få se alle endringene som trengtes.

- **KI skrev endringene** i fire filer: `GameApiClient`, `ServerGamesService`, PUT-endepunktet i `Program.cs` og `AdminGames.razor`.
- **KI kompilerte endringene** sammen med resten av prosjektet før de ble lagt frem. Appen ble ikke kjørt av KI.
- **KI forklarte hvorfor** hver endring trengtes. Forklaringene er grunnlaget for punktene under.
- **Innsetting og testing** av endringene i prosjektet ble ikke gjort av KI.

Kjeden for en oppdatering

Oppdateringen går gjennom fire lag. Hvert lag har ett ansvar, og svaret fra lagringen må sendes hele veien tilbake til siden.

Lag	Ansvar	Ved ukjent id	Ved tom kategori
`AdminGames.razor`	Leser teksten i raden og viser resultatet	Viser «Spillet finnes ikke lenger»	Stopper før kallet og viser melding
`GameApiClient`	Sender `PUT api/games/{id}` og tolker svaret	Gjør 404 om til `false`	Kaster unntak ved 400
Endepunkt i `Program.cs`	Oversetter mellom HTTP og C#	Svarer `404 Not Found`	Svarer `400 Bad Request`
`ServerGamesService`	Validerer på serveren	Sender `false` videre	Kaster `ArgumentException`
`GamesStorageAndQuery`	Finner og endrer spillet i lista	Returnerer `false`	Nås ikke

1. Svaret må sendes gjennom alle lagene

Lagringen vet om spillet fantes. Endepunktet trenger den informasjonen for å velge mellom 204 og 404. Derfor returnerer metoden `Task<bool>` i alle lagene, og hvert lag sender svaret videre i stedet for bare å vente på det.

ServerServices/ServerGamesService.cs

```
public async Task<bool> UpdateCategoryAsync(int id, UpdateCategoryRequest request)
{
    if (string.IsNullOrWhiteSpace(request.Category))
    {
        throw new ArgumentException("Kategori kan ikke være tom");
    }

    return await query.UpdateCategoryAsync(id, request.Category);
}
```

Lærdom: en `Task` uten verdi kan bare si «ferdig». Skal den som kaller vite **hvordan** det gikk, må resultatet være en del av returtypen.

2. Valideringsfeil skal bli 400, ikke 500

Tjenesten kaster `ArgumentException` når kategorien er tom. Fanges ikke unntaket, svarer ASP.NET Core med 500, som betyr at serveren har feilet. Det stemmer ikke: det er forespørselen som er ugyldig. Endepunktet fanger derfor unntaket og svarer 400.

Program.cs

```
gamesApi.MapPut("{id:int}", async (int id, ServerGamesService gamesService,
    UpdateCategoryRequest request) =>
{
    try
    {
        var updated = await gamesService.UpdateCategoryAsync(id, request);

        return updated ? Results.NoContent() : Results.NotFound();
    }

    catch (ArgumentException exception)
    {
        return Results.BadRequest(new { error = exception.Message });
    }
});
```

Lærdom: statuskoden forteller klienten **hvem** som har gjort feil. 4xx er klientens feil, 5xx er serverens. Valideringen gjøres på serveren selv om siden også sjekker, fordi API-et kan kalles utenom skjemaet, for eksempel fra en `.http`-fil.

3. Hver rad trenger sin egen tilstand

Med én felles variabel for ny kategori ville alle radene vist og endret samme tekst. Løsningen er en ordbok der id-en til spillet er nøkkelen. Feltet i hver rad leser og skriver bare sin egen verdi, og lagreknappen sender bare den radens id.

AdminGames.razor, @code

```
// New category typed in each row, keyed by game id
private readonly Dictionary<int, string> newCategories = new();

private string GetNewCategory(int id) =>
    newCategories.TryGetValue(id, out var category) ? category : string.Empty;
```

AdminGames.razor, tabellen

```
<td><input value="@GetNewCategory(game.Id)" @oninput="e => newCategories[game.Id] = e.Value?.
    ToString() ?? string.Empty" /></td>
<td><button @onclick="() => Update(game.Id)">Lagre</button></td>
```

Lærdom: når samme komponent eller markup gjentas i en løkke, må tilstand som tilhører én rad lagres per rad. Samme problem oppstår når to skjemaer bindes til samme modellobjekt.

4. Rekkefølgen på tilstandsendringer betyr noe

`LoadGames` setter `error = null` når hentingen lykkes. Settes meldingen «Spillet finnes ikke lenger» før lista lastes på nytt, slettes den med en gang, og brukeren ser den aldri. Meldingen settes derfor etter `LoadGames`.

```
AdminGames.razor, Update(int id)

var updated = await ApiClient.UpdateCategoryAsync(id, new UpdateCategoryRequest { Category =
    category });

newCategories.Remove(id);
await LoadGames();

// Set after LoadGames, which clears error
if (!updated)
{
    error = "Spillet finnes ikke lenger";
}
```

Lærdom: når flere metoder endrer samme felt, må det være kjent hva hver av dem gjør med det. Ellers kan én metode stille overskrive det en annen nettopp satte.

5. En feilmelding skal ikke skjule dataene

Feilmeldingen og tabellen sto opprinnelig i samme `if`/`else`-kjede, så en melding som «Kategori må fylles ut» fikk hele tabellen til å forsvinne. Nå vises meldingen over tabellen, og tabellen vises så lenge spillene er hentet. «Laster...» vises bare når det verken finnes data eller feil.

Lærdom: tilstandene laster, feil og tom liste utelukker ikke alltid hverandre. En feil ved oppdatering betyr ikke at lista er borte.

Begrensninger i KI-hjelpen

- KI kompilerte koden, men kjørte ikke appen. Oppførselen i nettleseren er ikke kontrollert av KI.
- Valideringen på siden sjekker bare at feltet ikke er tomt. Regler for hvilke kategorier som er gyldige, er ikke en del av endringen.